

AURIX: An Agentic Unified Risk Intelligence Platform

Final Year Project Progress Report (BCS-753)

Project Overview

Role: Backend & Infrastructure Engineer (Layer 2 & RAG Vector Infrastructure)

Student: Bhavya Gupta (Roll No: 2301641520064)

Branch: CS AI 3-A

Supervisor: Dr. Jyoti Dua (Associate Professor)

Weekly Technical Progress

Week	Work Done	Next Steps
Week 1	Provisioned cloud infrastructure using Supabase (Free Tier) and Upstash (Serverless Redis). Executed SQL schemas for core relational tables (USERS, PROJECTS, SCANS, and VERIFIED_VULNERABILITIES) including Foreign Key mapping.	Configure Supabase Authentication and implement JWT validation middleware.
Week 2	Configured Supabase "The Vault" with Email/Password and GitHub OAuth.	Initialize the Node.js Express API Gateway and deploy mock API endpoints.

Week	Work Done	Next Steps
	Implemented JWT validation middleware and Row Level Security (RLS) to restrict scan report visibility to project owners.	
Week 3	Initialized Node.js Express server. Developed mock endpoints (POST /api/scans/github and /upload) returning 202 Accepted status to enable parallel frontend and extension development.	Implement the mock GET status endpoint and deploy the server publicly.
Week 4	Developed GET /api/scans/{scan_id} mock endpoint returning schema-compliant JSON. Deployed Express server to Render/Railway for public teammate access.	Replace the mock GitHub ingestion route with real database and Redis logic.
Week 5	Integrated live backend logic for GitHub ingestion. Metadata is now saved to Supabase (PENDING status) and jobs are pushed to Upstash Redis Queue	Implement real multipart/form-data upload logic with Supabase Storage.

Week	Work Done	Next Steps
	(LPUSH) for async processing.	
Week 6	Integrated <code>multer</code> for multipart/form-data handling. Implemented Supabase SDK to stream .zip payloads directly to cloud storage buckets to manage memory constraints.	Implement the real database polling logic for the GET scan route.
Week 7	Refactored GET <code>/api/scans/{scan_id}</code> to perform live Supabase queries, returning real-time SCANNING or COMPLETED statuses and payloads.	Setup the pgvector database extension for the RAG infrastructure.
Week 8	Commenced Phase 3 (AI Upgrade). Enabled <code>pgvector</code> in Supabase and designed the <code>knowledge_base</code> table with high-dimensional vector columns for the RAG pipeline.	Develop the data seeding script (<code>seed_rag.js</code>) for OWASP security guidelines.
Week 9	Built <code>seed_rag.js</code> for document ingestion. Parsed OWASP Cheat Sheets, vulnerability PoC templates, and patch	Integrate Hugging Face inference to convert the ingested docs into vector embeddings.

Week	Work Done	Next Steps
	code templates for Python and JS.	
Week 10	Integrated <code>Transformers.js</code> with <code>all-MiniLM-L6-v2</code> model. Converted markdown security documentation into embeddings and populated the Supabase vector table.	Build the Similarity Search API (Threat Intel Endpoint) for the LangGraph node.
Week 11	Developed Threat Intel Similarity Search via Supabase RPC. Implemented cosine similarity search returning the top 3 relevant security contexts in under 500ms.	Create the internal webhook route to receive final payloads from the AI Worker.
Week 12	Initiated Phase 4 (Orchestration). Created POST <code>/api/internal/webhook/scan-complete</code> to serve as the catch mechanism for payloads from the AWS AI Worker.	Add database parsing and update logic within the webhook handler.
Week 13	Finalized webhook parsing logic to extract verified bugs, update	Implement API rate limiting to protect the

Week	Work Done	Next Steps
	SCAN status, and populate the VERIFIED_VULNERABILITIES relational table.	expensive AI compute resources.
Week 14	Implemented Redis-backed API Rate Limiting using <code>express-rate-limit</code> . Configured 429 error handling to prevent queue flooding and manage compute quotas.	Implement automated data sanitization via background cron jobs.
Week 15	Developed <code>node-cron</code> scheduled jobs for data sanitization. Implemented automatic deletion of .zip payloads from Supabase Storage post-scan to ensure privacy and storage efficiency.	Perform final end-to-end integration testing and platform optimizations.
Week 16	Conducted full-stack integration testing. Verified Redis queuing, webhook handshakes with AWS EC2, and RAG retrieval latency. Finalized production deployment.	Compile the final project report and prepare the architecture presentation.